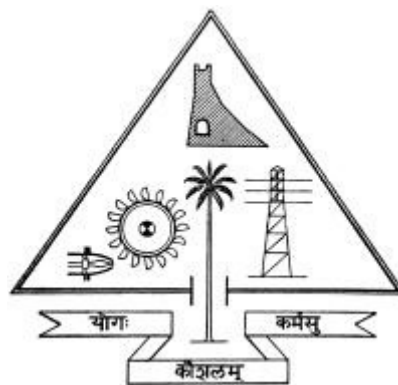


VISIONMATE: AN INTEGRATED AI-DRIVEN ASSISTIVE PLATFORM FOR VISUALLY IMPAIRED INDIVIDUALS

*Mini Project Report submitted in partial fulfillment of the requirements for the
award of the degree of Master of Computer Applications of the APJ Abdul
Kalam Technological University*

submitted by

**SILNA K DAISON
(TCR25MCA-2048)**



**DEPARTMENT OF COMPUTER APPLICATIONS
GOVERNMENT ENGINEERING COLLEGE
THRISSUR - 680009**

JULY 2026

ABSTRACT

Visual impairment presents significant daily challenges, often requiring individuals to use multiple assistive tools for reading, navigation, and digital information access. To address this, we introduce **VisionMate**, a voice-guided Android application that integrates AI-powered assistive technologies into a single platform for blind and visually impaired users.

VisionMate is built around two core modules. A **Braille Recognition and Digitization** pipeline uses image preprocessing, page detection, and a custom-trained Convolutional Neural Network (CNN) to segment and recognize Braille cells, converting physical Braille into structured digital text. A **Smart Digital Library** with Semantic Voice Search employs a lightweight MiniLM sentence-embedding model to generate vector embeddings, enabling users to retrieve documents through contextual voice queries using cosine similarity instead of file names.

The application also provides **OCR-based document reading** with contextual web assistance and a **YOLO-based Scene Description and Indoor Navigation** module for environmental awareness, obstacle detection, and navigation guidance. For user safety, a **gesture-triggered Emergency SOS** feature detects predefined phone shake patterns and automatically sends SMS alerts with GPS coordinates to emergency contacts.

VisionMate follows an offline-first architecture using optimized TensorFlow Lite models for fast, privacy-preserving on-device processing with minimal internet dependency. A fully voice-driven interface based on Automatic Speech Recognition (ASR) and Text-to-Speech (TTS), together with a lightweight device-based identity system, enables seamless interaction without visual input, promoting greater accessibility, independence.

CONTENTS

1	Introduction	1
1.1	Objectives	3
2	LITERATURE REVIEW	5
3	ENVIRONMENTAL STUDY	9
3.1	System Configuration	9
3.1.1	Hardware Requirements	9
3.1.2	Software Requirements	10
3.1.3	Software Requirements for Development	11
3.2	Software Specification	11
3.2.1	Dart	11
3.2.2	Python 3.8+	12
3.2.3	TensorFlow and Keras	12
3.2.4	TensorFlow Lite	12
3.2.5	OpenCV	13
3.2.6	YOLOv8	13
3.2.7	Sentence Transformers (MiniLM)	13
3.2.8	Google ML Kit OCR	14
3.2.9	Speech Recognition (speech_to_text)	14
3.2.10	Text-to-Speech (flutter.tts)	14
3.2.11	Camera Access (Flutter camera plugin)	14
3.2.12	Google Colab	15
4	SYSTEM ANALYSIS	16
4.1	Requirements Analysis	16
4.2	Existing System	17
4.3	Limitations of Existing System	18
4.4	Proposed System	18
4.5	Advantages of Proposed System	19
4.6	Functional Requirements	20
4.6.1	Braille Capture & Preprocessing	20
4.6.2	Braille Cell Segmentation & Classification	20
4.6.3	Semantic Document Search	20
4.6.4	OCR-Based Reading	21
4.6.5	Scene Description & Indoor Navigation	21
4.6.6	Emergency SOS	21
4.7	Non-Functional Requirements	22
4.7.1	Performance	22
4.7.2	Offline Functionality	22
4.7.3	Scalability	22
4.7.4	Accuracy	22
4.7.5	Reliability	22
4.7.6	Usability	23

4.7.7	Maintainability	23
4.8	Feasibility Study	23
4.8.1	Technical Feasibility	23
4.8.2	Operational Feasibility	23
4.8.3	Economic Feasibility	24
Bibliography		25

CHAPTER 1

INTRODUCTION

Visual impairment affects hundreds of millions of people worldwide, and for this population, everyday tasks that sighted individuals perform without conscious thought reading a printed letter, locating a stored document, or noticing an obstacle on a footpath — become significant sources of daily difficulty. Assistive technology has traditionally responded to this problem in fragments: Braille embossers handle reading and writing, dedicated OCR scanners handle printed-text conversion, GPS units handle outdoor navigation, and separate mobile apps handle object recognition or emergency alerting. A visually impaired user is therefore often forced to learn, carry, and switch between several specialized devices just to accomplish what would otherwise be a single, continuous task in daily life. The need for one unified, accessible platform has never been more pressing.

The core challenge lies in this fragmentation and its cost. Each additional device adds financial burden, a separate learning curve, and a dependency on being carried at exactly the right moment. This is particularly limiting in situations that demand a quick response, such as an emergency, where fumbling between multiple apps or devices can cost precious time. None of the existing tools, individually or together, offers a low-cost, voice-first experience that spans reading, organization, mobility, and safety. A number of existing systems already address parts of this problem, but each covers only a slice of it. General-purpose screen readers such as Google TalkBack make Android interfaces navigable by voice but offer no specialized support for Braille, document search, or scene understanding. AI-driven apps like Microsoft Seeing AI and dedicated hardware such as OrCam MyEye can read

printed text or recognize objects, but they depend heavily on cloud processing, raising concerns around latency, recurring data costs, and privacy, and neither converts physical Braille into an organized digital archive. Dedicated Braille note-takers and embossers digitize Braille effectively but are prohibitively expensive for most users and remain isolated from any broader reading or navigation workflow. Motion-based navigation aids and outdoor GPS apps such as BlindSquare assist with orientation but generate frequent false alerts from ordinary environmental clutter and are not designed for indoor use. Taken together, these systems show that the individual pieces of the puzzle already exist in the market, but no single, affordable, offline-first application yet brings them together under one voice-guided experience.

Recent advances in on-device deep learning have opened new possibilities for closing this gap. Lightweight Convolutional Neural Networks can now be trained to recognize fine-grained visual patterns, such as individual Braille cells, with high accuracy directly on a smartphone. Compact sentence-embedding models like MiniLM allow natural-language meaning to be captured as vectors small enough to run entirely offline, enabling search by context rather than by exact keywords. Real-time object detectors such as YOLO make it possible to describe a scene or flag an obstacle within a fraction of a second, while optimized TensorFlow Lite runtimes bring all of this inference onto the device itself, removing the need for constant connectivity or cloud processing. Together, these advances make it feasible to treat assistive technology not as a collection of single-purpose tools, but as one continuous, intelligent system built around the user's voice.

In this project, we propose **VisionMate**, a comprehensive, voice-guided Android application designed to bring reading, digital organization, navigation, and safety together under a single offline-first platform for blind and visually impaired users. The system is built around two core modules. A Braille Recognition and Digitization pipeline preprocesses a photographed Braille page, detects and segments individual cells, and classifies each one using a custom-trained CNN, converting physical Braille into structured,

searchable digital text. A Smart Digital Library then organizes this and other stored content using Semantic Voice Search: an on-device MiniLM model generates vector embeddings for every document, so a user's spoken query is matched to the most relevant document through cosine similarity rather than a remembered file name.

Around these two contributions, VisionMate integrates the everyday features a visually impaired user needs in one ecosystem: an OCR-based reader that expands recognized printed text with web-sourced context, a YOLO-based scene description and indoor navigation module that gives real-time obstacle alerts and general room overviews, and a gesture-triggered Emergency SOS system that detects a shake pattern and instantly sends an SMS with the user's GPS location to a trusted contact. Every AI model runs on-device through optimized TensorFlow Lite, so the application remains fully functional without a constant internet connection, and the entire experience from opening the app to retrieving a document to reporting an emergency is operated purely through speech, using automatic speech recognition and text-to-speech, with a lightweight, registration-free identity system.

1.1 Objectives

- To develop an AI-powered voice-guided Android application for visually impaired users.
- To recognize and digitize Braille documents using image processing and a custom-trained CNN.
- To implement a Smart Digital Library with semantic voice-based document search using MiniLM embeddings.
- To provide OCR-based document reading with text-to-speech and contextual information.
- To develop a YOLO-based scene description and indoor navigation system for obstacle detection and voice guidance.

- To implement a gesture-triggered Emergency SOS feature that sends SMS alerts with the user's GPS location.
- To provide a fully voice-driven interface using Automatic Speech Recognition (ASR) and Text-to-Speech (TTS).
- To ensure offline and low-latency operation using TensorFlow Lite models.
- To integrate reading, document management, navigation, and emergency assistance into a single accessible platform.

By combining accurate Braille digitization with meaning-based document retrieval, real-time scene awareness, and instant emergency alerting, VisionMate moves beyond the single-purpose assistive tools that came before it. It treats reading, organizing, moving, and staying safe not as separate problems but as one continuous experience guided entirely by voice. This offline-first, privacy-preserving design makes the system practical for daily, independent use — giving blind and visually impaired users a single, low-cost companion in place of a shelf of disconnected devices.

CHAPTER 2

LITERATURE REVIEW

Assistive technology for visually impaired individuals has historically developed as a collection of narrow, single-purpose tools rather than a unified system, and the literature surveyed for this project reflects that same fragmentation. Seven prior works were examined in detail, spanning Braille recognition, camera-based navigation, semantic document retrieval, optical character recognition, and emergency alerting, and each of these strands maps closely onto one of the technical modules proposed for VisionMate. Reviewing them cluster by cluster, rather than as an unordered list, makes it possible to see both how far each sub-domain has progressed on its own and how little effort has been directed toward combining these capabilities into a single, offline-capable, voice-driven platform.

Braille accessibility has received sustained attention from the computer vision community, and two works in this review address it from different technical angles. *Vamshi et al. (2024)*, [1], proposed a Convolutional Neural Network (CNN)-based approach for recognizing segmented Braille cells and converting them into machine-readable text. Their work demonstrated that a custom-trained CNN is a viable classifier for Braille cell recognition, which directly supports the classification approach adopted in VisionMate's own Braille pipeline. However, its scope stops at classification: it does not address text-to-speech output, multilingual handling, or on-device deployment, and it is in this sense a strict methodological subset of the recognition problem rather than a complete assistive solution. The stronger of the two, *Dholvan et al. (2025)*, [2], introduced an end-to-end Braille recognition framework

that combines CNN-based Braille recognition with speech synthesis and multilingual output. Building on the earlier classification-based approach, their system moves closer to a complete assistive solution by integrating text-to-speech functionality. Nevertheless, it does not organize recognized Braille content into a searchable digital repository or provide semantic retrieval capabilities, leaving significant opportunities for further enhancement.

A second cluster addresses independent mobility through camera-based scene understanding and navigation. *Dissanayake et al.(2021)*, [3] developed Navigate-Me, a secure indoor navigation system that relies on Bluetooth Low Energy (BLE) beacons for localization and voice-guided navigation. The system demonstrates that accurate indoor guidance can be achieved within beacon-equipped environments. However, its reliance on pre-installed BLE infrastructure limits its applicability in unfamiliar or unprepared spaces. In contrast, *Nwakeze (2026)*, [4] proposed a deep learning-based assistive navigation guide that employs a YOLOv8 object detection model to recognize obstacles and provide real-time navigation assistance without requiring external infrastructure. This work is the closest technical analog to VisionMate's Scene Description and Indoor Navigation module, which likewise relies on an on-device YOLO model rather than external beacons. Comparing these two approaches highlights a clear trade-off in the literature: beacon-based systems such as Navigate-Me can provide precise localization but only within pre-instrumented buildings, whereas camera-based systems such as the YOLOv8 Assistive Navigation Guide sacrifice some localization precision in exchange for operating in any environment without requiring external infrastructure. This trade-off favors the infrastructure-free design adopted in VisionMate, as blind users cannot rely on every building they enter being equipped with BLE beacons.

Digital document retrieval for visually impaired users depends on how effectively natural-language voice queries can be matched to stored content. *Korga et al. (2025)*, [5], investigated the performance of thirteen sentence embedding models, including MiniLM, for semantic similarity

tasks. Their comparative analysis demonstrated that embedding-based methods outperform traditional lexical and ontology-based approaches such as WordNet while maintaining lower computational requirements. These findings provide strong justification for VisionMate's adoption of the lightweight MiniLM model to enable efficient on-device semantic voice search. The paper does not, however, apply its findings to an accessibility context or to a voice-first retrieval interface, which is where VisionMate's Smart Digital Library extends the existing work.

Printed document accessibility has also been extensively studied. *Kirana et al. (2025)*, [6] compared the performance of Tesseract OCR, EasyOCR, and Transformer-based OCR models for handwritten text recognition. Their evaluation highlights the trade-offs between recognition accuracy, computational complexity, and inference speed among different OCR engines. These findings provide valuable guidance for selecting an OCR solution suitable for mobile deployment in VisionMate's document reading module.

The final cluster concerns user safety. *Harish et al. (2025)*, [7] presented a mobile-based emergency response system that automatically dispatches alerts with real-time location information to predefined contacts during emergency situations. Their system supports multiple activation methods, including a single tap, voice commands, and shake gestures, together with GPS-based location sharing, making it the closest existing analog to VisionMate's Emergency SOS feature. However, the proposed system primarily focuses on emergency communication and does not integrate this functionality with broader assistive features such as Braille digitization, semantic document search, OCR-based document reading, or AI-driven scene understanding. VisionMate extends this work by incorporating a shake-gesture-triggered SOS module within a unified assistive platform for visually impaired users, enabling rapid SMS alerts with GPS coordinates alongside comprehensive accessibility services.

The reviewed studies demonstrate that key technologies such as Braille recognition, OCR-based reading, semantic document search, scene

understanding, indoor navigation, and emergency alerting have been successfully developed as independent solutions. However, none of the existing research integrates these capabilities into a single, unified assistive platform. Most systems are designed for specific tasks, rely on separate interfaces or hardware, and lack seamless interaction between modules. Furthermore, few solutions emphasize offline, on-device processing for improved privacy and accessibility. This research gap motivates the development of VisionMate, which integrates these proven technologies into a single offline-first, voice-driven Android application, providing a comprehensive and user-friendly assistive ecosystem for visually impaired individuals.

Therefore, the next chapter presents the environmental study undertaken to identify the tools, frameworks, and device constraints adopted for implementing VisionMate.

CHAPTER 3

ENVIRONMENTAL STUDY

This chapter describes the hardware and software environment required for the development and implementation of *VisionMate: An Integrated AI-Driven Assistive Platform for Visually Impaired Individuals*. The environment supports the integration of AI technologies such as Braille recognition, semantic search, OCR, scene understanding, indoor navigation, and emergency SOS within a single Android application.

The project is developed using open-source frameworks and standard development tools, ensuring a cost-effective, scalable, and efficient implementation. The selected technologies support AI model development, mobile application deployment, and offline processing, enabling the system to provide reliable assistance for visually impaired users.

3.1 System Configuration

System configuration describes the hardware and software requirements for developing, training, testing, and deploying the proposed VisionMate system. The application combines Android application development, computer vision, machine learning, natural language processing, and speech technologies. Google Colab is used for training deep learning models, while Android Studio is used for application development and deployment.

3.1.1 Hardware Requirements

- **Processor:** Intel Core 3 (11th Generation or higher) or AMD Ryzen 5 equivalent

- **Memory (RAM):** 8 GB minimum (16 GB recommended)
- **Storage:** Minimum 30 GB free SSD space
- **GPU:** Optional for local development; Google Colab T4 GPU used for CNN model training
- **Internet Connection:** Required for downloading datasets, SDKs, pre-trained models, and Google Colab access
- **Android Device:** Android 10 or above with camera, microphone, speaker, GPS, and accelerometer

3.1.2 Software Requirements

- **Operating System:** Windows 10/11, Ubuntu 22.04 LTS, or macOS
- **App Development Framework:**Flutter SDK
- **IDE:** Visual Studio Code with flutter/DART plugins
- **Programming Languages:** Dart (app) + Python (training)
- **Machine Learning Framework:** TensorFlow and Keras
- **Deployment Framework:** TensorFlow Lite
- **Computer Vision Library:** OpenCV
- **Object Detection Model:** YOLOv8 (Ultralytics)
- **Semantic Search Model:** all-MiniLM-L6-v2 (Sentence Transformers)
- **OCR Engine:** Google ML Kit Text Recognition
- **Speech Recognition:** Android SpeechRecognizer API
- **Text-to-Speech:** Android TextToSpeech API
- **Development Environment:** Google Colab and Jupyter Notebook

3.1.3 Software Requirements for Development

In addition to the execution-time software, several development tools are required to design, implement, test and maintain the AssistAI application. These tools facilitate mobile application development, model training, debugging, version control and experimentation.

The key development software requirements are detailed as follows:

- **Operating System:** Windows 10/11, Ubuntu 22.04 LTS or macOS.
- **Front End:** Flutter SDK with Dart.
- **Back End:** Python 3.11 or later for machine learning development and experimentation.
- **IDE Used:** Visual Studio Code with Flutter and Dart extensions.
- **Version Control:** Git.
- **Machine Learning Environment:** Google Colab and Jupyter Notebook.
- **Testing Tools:** Flutter Testing Framework and Android Emulator.
- **Documentation Tools:** LATEX for preparing project reports and documentation.
- **Visualization:** Matplotlib for analysing and visualizing experimental results.

3.2 Software Specification

Software specification describes the programming languages, frameworks, libraries, and development tools used to implement the VisionMate system.

3.2.1 Dart

Dart is the primary programming language used for developing the VisionMate application. It is the language used by Flutter, Google's

cross-platform UI toolkit, and offers concise syntax, strong null safety, and a hot-reload development cycle that speeds up testing on real devices. Dart is responsible for implementing the application's voice-driven navigation logic, camera interaction, coordination between the on-device AI modules (Braille CNN, YOLO, MiniLM), the Emergency SOS trigger, and communication with Flutter plugins for speech, location, and SMS.

3.2.2 *Python 3.8+*

Python is selected as the language for model training and dataset preparation due to its extensive ecosystem for data science and deep learning. Its simplicity and readability enable rapid experimentation, while libraries such as TensorFlow, Keras, and OpenCV provide the necessary building blocks for image preprocessing, CNN training, and model conversion. Python serves as the backbone of VisionMate's model-development pipeline, handling dataset loading, Braille cell image preprocessing, CNN training and validation, and conversion of the final trained model into TensorFlow Lite format for deployment on Android.

3.2.3 *TensorFlow and Keras*

TensorFlow, together with its high-level Keras API, is used to design, train, and evaluate the Convolutional Neural Network (CNN) responsible for Braille character recognition. The framework provides optimized neural network layers, GPU acceleration during training, and built-in tools for data augmentation and validation, allowing the model to learn robust visual features from the labeled Braille cell dataset. Once training is complete, the resulting model is converted into TensorFlow Lite format so that it can be deployed and executed directly on Android devices.

3.2.4 *TensorFlow Lite*

TensorFlow Lite is the runtime used for efficient on-device execution of the trained CNN model. Through post-training quantization, the converted

model is compressed and optimized for mobile hardware, enabling the Braille recognition pipeline to run directly on the smartphone without requiring cloud connectivity. This ensures faster inference, improved user privacy, and full offline functionality, in line with the offline-first design goals of VisionMate.

3.2.5 OpenCV

OpenCV (Open Source Computer Vision Library) is used for all image preprocessing tasks required by the Braille recognition pipeline. It performs page detection and perspective correction on a photographed Braille page, applies noise reduction and contrast enhancement to compensate for inconsistent lighting, and segments the corrected image into individual Braille cells before each cell is passed to the CNN classifier. OpenCV's efficiency and extensive documentation make it a reliable choice for the image-processing stage of the pipeline.

3.2.6 YOLOv8

YOLOv8 (You Only Look Once, Version 8) is used for real-time object detection and scene understanding in the Scene Description and Indoor Navigation module. The model detects surrounding objects, obstacles, doors, furniture, and other environmental elements from the live camera feed. Detected objects are converted into short spoken descriptions through the text-to-speech engine, assisting visually impaired users in understanding their surroundings and navigating indoor spaces safely.

3.2.7 Sentence Transformers (MiniLM)

The Sentence Transformers library, using the lightweight all-MiniLM-L6-v2 model, powers the semantic document search feature of the Smart Digital Library. Stored documents and spoken user queries are each converted into dense vector embeddings, and cosine similarity is computed between them to retrieve the most semantically relevant documents. This allows users to

locate content based on meaning and context rather than relying on exact keyword or file-name matches.

3.2.8 Google ML Kit OCR

Google ML Kit Text Recognition is used to extract text from printed documents captured through the smartphone camera. The recognized text is passed to the text-to-speech engine for immediate playback, enabling users to independently read printed material such as letters, labels, and notices without sighted assistance.

3.2.9 Speech Recognition (*speech_to_text*)

Voice input is handled through the `speech_to_text` Flutter plugin, which wraps the native Android `SpeechRecognizer` API. It converts the user's spoken commands into text, allowing them to navigate the application, issue search queries, and activate specific features entirely through voice, without any reliance on a visual interface.

3.2.10 Text-to-Speech (*flutter_tts*)

Spoken feedback is generated through the `flutter_tts` plugin, which wraps the native Android Text-to-Speech engine. It converts textual information into natural speech output, providing voice feedback for OCR results, document search results, navigation guidance, scene descriptions, and general application notifications.

3.2.11 Camera Access (*Flutter camera plugin*)

The smartphone camera is accessed through Flutter's camera plugin, which uses `CameraX` internally on Android to provide a stable, consistent camera interface across different device models. It is used to capture photographs of Braille documents and printed text, as well as to stream live video for the Scene Description and Indoor Navigation module.

3.2.12 Google Colab

For GPU-accelerated training, Google Colab is used to train and validate the Braille recognition CNN, providing free access to T4 GPUs along with pre-installed deep learning libraries. The notebook format allows the training pipeline to be executed step by step, making it easy to debug issues, visualize intermediate results such as segmented Braille cells and training accuracy curves, and export the final trained model for conversion to TensorFlow Lite.

The environmental study defines the hardware and software requirements necessary for the successful development of VisionMate. The selected technologies provide an efficient, lightweight, and offline-first environment that supports all major system functionalities, including Braille recognition, semantic search, OCR, scene understanding, indoor navigation, and emergency assistance. By utilizing on-device AI models and native Android services, the proposed environment ensures privacy, reliability, cost-effectiveness, and ease of deployment, making VisionMate a practical and accessible assistive solution for visually impaired users.

CHAPTER 4

SYSTEM ANALYSIS

System analysis for VisionMate examines the multi-module, on-device AI pipeline that transforms physical Braille pages, printed documents, and live surroundings into accessible speech and structured, searchable data. It maps the complete workflow from image or voice input to spoken output, ensuring reliable offline operation across Braille recognition, semantic document search, scene understanding, and emergency alerting.

This systematic approach delivers accurate Braille digitization, meaning-based document retrieval, real-time obstacle awareness, and instant safety response for blind and visually impaired users, built on established Flutter, TensorFlow Lite, OpenCV, and on-device Transformer technologies.

4.1 Requirements Analysis

Visual impairment affects hundreds of millions of people worldwide, and existing assistive tools address only fragments of their daily needs. A user may own a Braille note-taker, a separate OCR reading app, a navigation aid, and a manual emergency contact process – each learned and operated independently. This fragmentation increases cost, cognitive load, and the risk of critical delay during an emergency.

The core functional requirement is a unified, voice-guided Android application capable of digitizing physical Braille into searchable text, retrieving stored documents by spoken meaning rather than file name, reading printed material aloud with contextual expansion, describing surroundings and flagging obstacles in real time, and dispatching emergency alerts automatically on a shake gesture – all without requiring a visual

interface or constant internet connectivity.

Beyond individual features, the system addresses an operational imperative: every module must run reliably on consumer-grade Android hardware, respond quickly enough for real-time use (obstacle alerts, emergency dispatch), and preserve user privacy by keeping personal documents and images on-device wherever possible.

4.2 Existing System

Currently, assistive support for blind and visually impaired users typically relies on:

- **Screen Readers (e.g., TalkBack):** General-purpose accessibility tools that read on-screen UI elements aloud, but offer no specialized support for Braille digitization, document search, or scene understanding.
- **Dedicated Braille Hardware:** Braille embossers and note-takers digitize Braille effectively but are prohibitively expensive and remain isolated from any broader reading, search, or navigation workflow.
- **Cloud-Based OCR/Object Recognition Apps (e.g., Seeing AI, OrCam):** Capable of reading text or recognizing objects, but dependent on continuous cloud connectivity, introducing latency, data costs, and privacy exposure.
- **Motion-Based Navigation Aids:** Basic obstacle-detection tools that trigger on any movement, generating frequent false alerts from ordinary environmental clutter such as swaying curtains or passing shadows.
- **Manual Emergency Procedures:** Locating a phone, unlocking it, and manually dialing or messaging a contact during an emergency – a process that can be slow or impossible for a visually impaired user under stress.

4.3 Limitations of Existing System

- **Fragmentation:** No single tool spans reading, document organization, mobility, and safety – users must learn and carry multiple devices.
- **Lack of Semantic Understanding:** File-based digital libraries require users to remember exact file names, offering no way to search by meaning or context.
- **Cloud Dependency:** Many existing tools require constant internet access, making them unreliable in poor network conditions and raising privacy concerns over transmitted images.
- **High False-Alert Rates:** Motion-based navigation aids cannot distinguish a genuine obstacle from harmless environmental movement.
- **Slow Emergency Response:** Manual emergency procedures cost precious time exactly when speed matters most.
- **High Cost:** Dedicated Braille hardware remains financially out of reach for many users.

Existing assistive technologies successfully address individual tasks such as reading, navigation, and emergency support. However, they operate independently, requiring users to rely on multiple applications or devices. This fragmentation, together with cloud dependency, high cost, and limited integration, highlights the need for a unified and accessible assistive platform such as VisionMate.

4.4 Proposed System

The proposed system, VisionMate, uses a multi-module, on-device AI architecture combining computer vision, semantic language understanding, and speech technologies within a single Flutter-based Android application. It replaces isolated single-purpose tools with one continuous, voice-guided

experience. The pipeline includes image preprocessing and page detection via OpenCV, Braille cell segmentation and classification via a custom-trained CNN, semantic document embedding and retrieval via MiniLM, printed-text extraction via Google ML Kit OCR, real-time object detection via YOLOv8, and gesture-triggered emergency alerting via device sensors and SMS. The system produces spoken output at every stage through Android's text-to-speech engine, with all core AI inference running offline via TensorFlow Lite.

4.5 Advantages of Proposed System

- **Unified Experience:** Combines reading, document organization, navigation, and safety into a single application, eliminating the need for multiple specialized tools.
- **Semantic Search:** MiniLM-based embeddings allow documents to be retrieved by meaning, not exact file names, through natural spoken queries.
- **Offline-First Design:** Core AI modules (Braille CNN, YOLOv8, MiniLM) run entirely on-device via TensorFlow Lite, ensuring functionality without constant connectivity and reducing privacy exposure.
- **Context-Aware Alerts:** YOLOv8's object-level detection reduces false alerts compared to simple motion detection, distinguishing real obstacles from background movement.
- **Rapid Emergency Response:** Shake-gesture detection removes the need to unlock or navigate a phone manually, sending SMS alerts with GPS coordinates within seconds.
- **Low Cost:** Built entirely on free and open-source tools and standard smartphone hardware, avoiding the cost of dedicated Braille or navigation hardware.

- **Accessible by Design:** A fully non-visual, voice-driven interface (ASR/TTS) ensures every feature is usable without sighted assistance.

4.6 Functional Requirements

Functional requirements are detailed descriptions of what the system should do and how it should behave in various situations.

4.6.1 Braille Capture & Preprocessing

- The system shall accept a photograph of a physical Braille page via the device camera.
- It shall detect the page boundary and correct perspective distortion using OpenCV.
- It shall enhance contrast and reduce noise to compensate for inconsistent lighting.

4.6.2 Braille Cell Segmentation & Classification

- The preprocessed image shall be segmented into individual Braille cells.
- Each cell shall be classified into its corresponding character using the custom-trained CNN.
- Classified characters shall be assembled into structured, readable digital text.

4.6.3 Semantic Document Search

- Stored documents shall be converted into vector embeddings using the MiniLM model.
- Spoken user queries shall be converted into text via ASR, then embedded in the same vector space.

- Cosine similarity shall be computed between the query embedding and stored document embeddings to rank and return the most relevant results.

4.6.4 OCR-Based Reading

- The system shall accept a photograph of printed text via the device camera.
- Google ML Kit shall extract the text content from the image.
- Recognized text shall be read aloud via text-to-speech, with key terms optionally expanded using web-sourced context when internet access is available.

4.6.5 Scene Description & Indoor Navigation

- The system shall continuously analyze the live camera feed using YOLOv8 to detect objects and obstacles.
- Detected obstacles within a critical distance shall trigger immediate spoken alerts.
- On request, the system shall provide a general spoken overview of the surrounding environment.

4.6.6 Emergency SOS

- The system shall continuously monitor accelerometer data for a defined shake pattern.
- On detecting the pattern, it shall retrieve the user's current GPS coordinates.
- It shall automatically send an SMS alert containing the coordinates to a pre-configured emergency contact.

4.7 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system rather than specific behaviors, ensuring the application is efficient, robust, and usable under real-world conditions.

4.7.1 Performance

The system shall classify a full Braille page and return digital text within a few seconds on mid-range Android hardware, and shall issue obstacle alerts with minimal perceptible delay during live navigation.

4.7.2 Offline Functionality

All core AI modules (Braille CNN, YOLOv8, MiniLM) shall function fully offline via TensorFlow Lite; only OCR's contextual web expansion shall require an internet connection.

4.7.3 Scalability

The Smart Digital Library's local database shall support a growing number of stored documents without significant degradation in search response time.

4.7.4 Accuracy

The Braille CNN shall achieve high classification accuracy across standard six-dot Braille patterns under reasonably controlled lighting conditions.

4.7.5 Reliability

The system shall handle minor variations in image quality, lighting, and camera angle gracefully, and the Emergency SOS module shall reliably trigger despite background app state.

4.7.6 Usability

The system shall be operable entirely through voice, with no reliance on visual interaction, and shall provide clear spoken feedback at every step for a blind or visually impaired user.

4.7.7 Maintainability

The codebase shall be modular, separating the Braille pipeline, semantic search, OCR, scene description, and SOS modules, allowing individual models to be retrained or upgraded independently.

4.8 Feasibility Study

The VisionMate project demonstrates feasibility across technical, operational, and economic dimensions, leveraging mature, freely available technologies for practical deployment on consumer Android devices.

4.8.1 Technical Feasibility

The project uses well-established frameworks – TensorFlow/Keras for training, TensorFlow Lite for on-device inference, OpenCV for image processing, and pre-trained models (YOLOv8, MiniLM) – all of which run efficiently on standard Android hardware and can be trained using free GPU access via Google Colab, requiring no specialized infrastructure.

4.8.2 Operational Feasibility

The system integrates as a single, self-contained Android application with a fully voice-guided interface, requiring no technical expertise and no complex registration process from end users, making adoption straightforward for blind and visually impaired individuals of varying technical backgrounds.

4.8.3 Economic Feasibility

Built entirely on open-source frameworks and standard smartphone hardware, the solution incurs no licensing costs and requires no dedicated Braille or navigation hardware. Primary value stems from consolidating multiple paid assistive tools into a single free application, reducing both financial burden and the learning curve for independent daily use.

BIBLIOGRAPHY

- [1] B. Vamshi, B. Deekshitha, Gunda Snehitha, Rashmika Bhandaru and Indira Priyadarshini, "Braille Recognition and Conversion to Text Using CNN Technique," *2024 Intelligent Systems and Machine Learning Conference (ISML)*, IEEE, pp. 474–478, 2024. doi: 10.1109/ISML60050.2024.11007406.
- [2] Mohan Dholvan, Y. Mohanalaxmi, Shailaja Mantha, R. Keerthana and K. Varunkumar, "Revolutionizing Braille with Speech Synthesis," *2025 11th International Conference on Smart Computing and Communications (ICSCC)*, IEEE, 2025. doi: 10.1109/ICSCC66177.2025.11233490.
- [3] D. M. L. V. Dissanayake, S. A. D. S. P. Solanga, R. G. M. D. R. P. Rajapaksha, J. A. D. C. Anuradha Jayakody and U. P. Prabhashawara, "Navigate-Me: Secure Voice Authenticated Indoor Navigation System for Blind Individuals," *2021 21st International Conference on Advances in ICT for Emerging Regions (ICTer)*, IEEE, pp. 219–224, 2021. doi: 10.1109/ICTer53630.2021.9774790.
- [4] Osita Miracle Nwakeze, "An Intelligent Assistive Navigation Guide System for Blind and Visually Impaired Individuals Using Deep Learning Techniques," *International Journal of Engineering & Tech Development (IJETD)*, vol. 2, no. 2, 2026. doi: 10.63665/IJETD-Y2F2A001.
- [5] Alex Maximilian Korga, Sebastian Wefers, Keno Hanken, Raad Bin Tareaf, Ben Steemers and Hunaida Awwad, "Does Size Matter? Examining Sentence Similarity Performance in Large Language Models," *2025 International Conference on Information Networking (ICOIN)*, IEEE, 2025. doi: 10.1109/ICOIN63865.2025.10993176.

- [6] Kartika Candra Kirana, Afwatul Maqbullah, Ira Kumalasari, Agung Faradiz Shobari, Gulpi Qorik Oktagalu and Bagus Hidayat, "Comparison of Tesseract OCR, Easy OCR, and Transformer OCR on Handwritten Image," *2025 9th International Conference on Electrical, Electronics and Information Engineering (ICEEIE)*, IEEE, 2025. doi: 10.1109/ICEEIE66203.2025.11252079.
- [7] N. Harish, V. Karunya Sri, R. Kamaladharshini, J. Jeyasri and K. Praveena, "SOS-Save Our Soul Alert App: An Emergency Response System with Real-Time Location Sharing," *International Journal of Research Publication and Reviews*, vol. 6, no. 11, pp. 8258–8269, 2025.